

Lecture 19

Thursday April 16th

Pintos Alarm Clock

- current implementation is doing a busy wait and this needs to be updated to do a sleep - wakeup
- `ready` and `running` are the only implemented states. There is a `block` but it is not implemented
- when a thread moves from ready to running it runs the check time and if waiting moves to another process.
- without an alarm function each process is left to constantly check the clock and then move to the back of the queue if its not their time.
- The solution is to put them all to sleep and then wake them up when it is time
- `Timer_sleep()`
 - put the threads into a sleep state
 - right now all processes are in the `ready_list`
 - need to create a `sleep_list` to put the threads into
- Alarm clock needs to have a function `timer_interrupt()` that will keep track of the time and when its time it will wake up the corresponding process and put it in the `ready_list`
- `local_tick` needs to be created so that each thread knows what time it needs to be woken up. This needs to be added to the struct

Deadlocks

- with single resource cycle detection will tell you if there is a deadlock
- multiple resource instance cycle detection will not work so you will need something like the bankers algorithm
 - given a snapshot of the system you need to look if there is a sequence that can run
 - if even 1 sequence can run you are fine
 - Run the bank request on the requested resource for the process and if it passes that check run the safety algorithm on the other values to see if there are issues